中文完整講解 / MATLAB附錄改編

從陣列到時間序列，用Python讀懂量化計算

Ernest P. Chan | Quantitative Trading | 第二版2021

原書附錄介紹MATLAB如何用陣列、資料表與工具函式支援量化工作。本版保留其學習範圍，改用Python、NumPy、pandas與Matplotlib，逐步把日期、價格、股數及輸出連在一起。

讀完應能完成的工作

把日期、價格與股票名稱放入合適物件，說清楚型別與shape。

由一個數字代入，讀懂切片、布林篩選、廣播與矩陣乘法。

辨別價格差、美元損益、報酬率、平均及標準差的不同單位。

保留資料時間、缺值與初始條件，不把循環位移當成前收。

按日期合併、彙整與繪圖，對回相同期間的可重算結果。

不要求先會MATLAB；也不假設教師已熟NumPy。每題提供必要輸入、可單獨執行程式及真實輸出；用本書前八章的財務問題理解工具，而不是重新教授整套投資策略。

原創教學改編，非官方譯本或逐句翻譯。原書、私人討論答案與完整資料檔不隨本PDF公開。

怎樣執行，數字從哪裡來？

執行環境

本版實跑Python 3.12.14、NumPy 2.2.6、pandas 2.2.3、Matplotlib 3.10.8。將某題程式整段放入.py檔或一個notebook儲存格，選同一Python環境後執行。每題有自己的import與輸入，不需讀取本機私人路徑或先下載行情。缺套件時，先確認選到哪個環境；本教材不要求付費服務、管理員權限或GPU。

共用的真實價格

2024交易日	AAPL Close	MSFT Close
01-02	185.64	370.87
01-03	184.25	370.60
01-04	181.91	367.94
01-05	181.18	367.75
01-08	185.56	374.69
01-09	185.14	375.79

價格單位均為美元 / 股，沿用前章保存歷史檔的原始Close，以Decimal四捨五入至美分；不是Adj Close，也不是2024當時封存、逐筆報價或可成交證據。所選六日無股利 / 分割事件。

假設與語法數字

10股 / 5股、1,100美元預算、配置事件及費用是明示教學設定，不是真實帳戶。亂數與取整中點只是程式物件，沒有冒充市場行情；沒有另造股價或收益序列。

閱讀順序

1-7先建立容器、索引與相鄰運算；8-16處理矩陣、形狀及型別；17-27計算、篩選及寫函式；28-32核對時間、合併、彙整及圖形。後面完整列出原書50項函式的Python方向與限制，不強迫所有工具一對一對應。

ISSUE 01 / Python附錄

Python、NumPy與pandas各負責什麼？

先把日期、數字與帶欄名資料放對物件，才知道程式正在操作什麼。

原書用MATLAB的語言、編輯器、除錯與陣列工具說明如何支援回測。本版改用Python語言：清單存日期，NumPy處理數值陣列，pandas的Series把每個價格接回日期。它們不是三套不同價格，也不自動形成策略。

本例AAPL 2024/1/2、1/3、1/4 Close依序185.64、184.25、181.91美元 / 股。三筆平均 = (185.64 + 184.25 + 181.91) / 3≈183.9333美元 / 股；不是三日報酬。



同三筆保存收盤價的三種工作；不是三個預測模型

執行前：先看：三筆平均會介於181.91與185.64之間嗎？輸出是否保留每個日期？

```
import numpy as np
import pandas as pd
dates = ["2024-01-02", "2024-01-03", "2024-01-04"]
close = np.array([185.64, 184.25, 181.91])
prices = pd.Series(close, index=dates, name="AAPL_USD/share")
print(prices.to_string())
print("Python values:", len(dates), "NumPy mean:", close.mean())
```

實際執行輸出

```
2024-01-02    185.64
2024-01-03    184.25
2024-01-04    181.91
Python values: 3 NumPy mean: 183.9333333333333
```

輸出三個日期都仍對到原價，平均約183.9333。將下方程式整段放入Python檔或一個notebook儲存格執行；每題都含自己的import與輸入，可單獨重跑。#是註解；print才明確列印結果，不用MATLAB的分號控制顯示。

討論

若只留下三個價格、刪掉日期標籤，仍能算平均嗎？哪項核對反而做不到？

原書印199-200 / PDF223-224；Python重組見[1]，本例為保存AAPL Close。

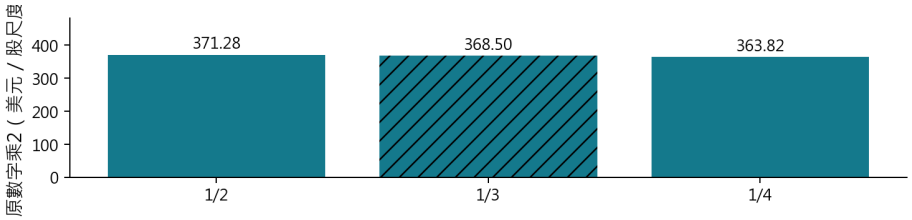
ISSUE 02 / Python附錄

一串收盤價，為何用NumPy陣列？

相同的乘號，對Python清單與數值陣列並不是同一種操作。

list是Python清單；np.array(...,dtype=float)建立可逐元素計算的浮點數陣列。float64表示本例的數值儲存型別，不表示價格精確到64位小數；shape為(3,)表示一維、含三個值，不是三列一欄。

仍用1/2-1/4 AAPL三價，純運算示範乘2：第一個數應是185.64×2 = 371.28。清單乘2只是將整串重複兩次，不是把三個價格加倍；本例不假設市場股價翻倍。



2024三筆真價的算術轉換；不是另一段市場行情

執行前：乘2後，清單與陣列各會有幾個數？第一個陣列結果是多少？

```
import numpy as np
close_list = [185.64, 184.25, 181.91]
close = np.array(close_list, dtype=float)
doubled = close * 2
print("list * 2:")
print(close_list * 2)
print("array * 2:", doubled.tolist())
print("dtype:", close.dtype, "shape:", close.shape)
```

實際執行輸出

```
list * 2:
[185.64, 184.25, 181.91, 185.64, 184.25, 181.91]
array * 2: [371.28, 368.5, 363.82]
dtype: float64 shape: (3,)
```

list輸出六個數，array輸出三個數371.28、368.50、363.82。可見先確認物件型別，才判斷運算是否回答原問題；只看程式沒有報錯還不夠。

討論

若想得到一股AAPL每個日期「價格加1美元」的結果，對close_list直接寫+1適合嗎？應選哪個物件？

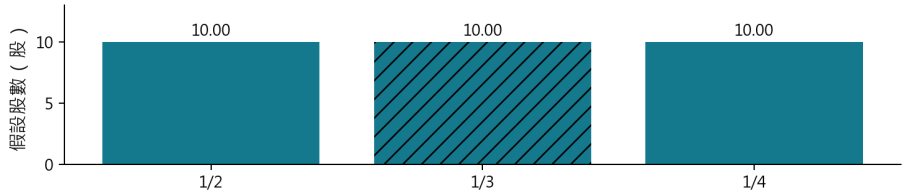
原書印200-201 / PDF224-225的陣列與運算；Python差異見[1]。

要放三天的股數，怎樣先建立容器？

初始化可以宣告待放數字的位置；不能把預先填0當成已觀察到零損益。

np.full(3,10.0)建立三個10；np.ones(3,dtype=int)建立三個整數1；np.zeros(3)建立三個0。本例三個位置預定對應2024/1/2、1/3、1/4，10股是另外設定的持有假設，不是行情資料。

原書用ones建立相同初值，本版直接用full表達「每個位置先放10」。cash_changes只是尚未計算的現金變動容器；它還沒接到買款、費用或股利，不能拿三個0宣稱帳戶沒變。



三日固定10股的設定；不是由價格估得的部位

執行前：full、zeros、ones輸出的三組數分別是多少？

```
import numpy as np
shares = np.full(3, 10.0)
cash_changes = np.zeros(3)
one_each = np.ones(3, dtype=int)
print("assumed shares:", shares.tolist())
print("initial cash-change container:", cash_changes.tolist())
print("ones:", one_each.tolist())
```

實際執行輸出

```
assumed shares: [10.0, 10.0, 10.0]
initial cash-change container: [0.0, 0.0, 0.0]
ones: [1, 1, 1]
```

輸出股數[10,10,10]、初始容器[0,0,0]和ones的[1,1,1]各有不同意思。NumPy陣列不能像原書MATLAB逐項賦值那樣，向不存在的第4個位置自動長大；應明確建立需要的形狀。

討論

若要先保留6個交易日的待算現金變動位置，應建立什麼？這六個0能先用來算策略績效嗎？

原書印200 / PDF224的ones與初始化；Python建立陣列見[1]。

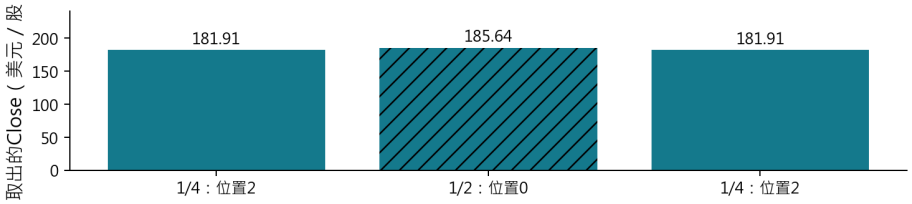
ISSUE 04 / Python附錄

MATLAB第1筆，Python要寫哪個位置？

按位置取價時，先把人說的第幾筆對回從0開始的索引。

AAPL 2024/1/2、1/3、1/4的價格185.64、184.25、181.91，在Python位置為0、1、2。MATLAB x(1)對應Python close[0]；close[2]是第3筆，不是第2筆。

positions=[2,0,2]要求依序取第3、第1、第3筆，因此應得181.91、185.64、181.91；順序可以重排，也可以重複，不會自動按日期排序。日期也必須用同一組positions選取。



同一位置規則同時選日期與價格；重複選取不代表出現新交易日

執行前：[2,0,2]會輸出哪三個日期？重複位置會被去掉嗎？

```
import numpy as np
dates = np.array(["2024-01-02", "2024-01-03", "2024-01-04"])
close = np.array([185.64, 184.25, 181.91])
positions = np.array([2, 0, 2])
selected = close[positions]
print("positions:", positions.tolist())
print("dates:", dates[positions].tolist())
print("Close_USD/share:", selected.tolist())
```

實際執行輸出

```
positions: [2, 0, 2]
dates: ['2024-01-04', '2024-01-02', '2024-01-04']
Close_USD/share: [181.91, 185.64, 181.91]
```

輸出日期依序1/4、1/2、1/4，價格依序181.91、185.64、181.91。若只重排價格而保留原日期，數值都看似合理，但每筆的時間已錯。

討論

若想依序選1/3再1/2，positions應寫什麼，得到哪兩個價格？

原書印201 / PDF225的索引重排；原文將z的選取誤稱y，本版按實際運算物件說明。見[2]。

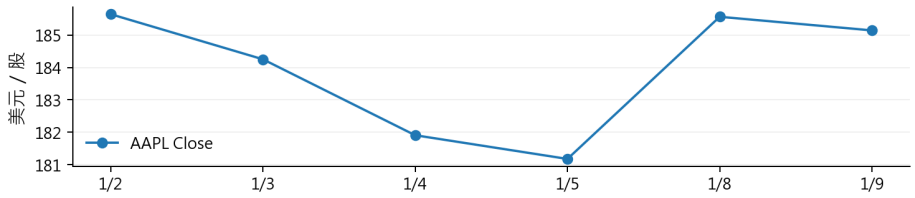
ISSUE 05 / Python附錄

取前三個交易日，為何右端寫3？

Python切片的右端不包含在結果中；取日期區間不能只翻譯原來的數字。

start:stop取位置start至stop之前，所以close[:3]取0、1、2，得到1/2、1/3、1/4；不是包含位置3的1/5。省略start即由開頭取。

close[1:4]改取1、2、3，對應1/3、1/4、1/5，價格184.25、181.91、181.18美元 / 股。右端減左端4-1＝3，是此步長為1且範圍有效時的筆數。



2024保存六交易日；[:3]選前3點，[1:4]將視窗右移一筆

執行前：[:3]與[1:4]各有幾筆？兩者的起點與終點是否相同？

```
import numpy as np
dates = np.array(["2024-01-02", "2024-01-03", "2024-01-04",
                  "2024-01-05", "2024-01-08", "2024-01-09"])
close = np.array([185.64, 184.25, 181.91, 181.18, 185.56, 185.14])
first_three = close[:3]
print("[:3] dates:", dates[:3].tolist())
print("[:3] prices:", first_three.tolist())
print("[1:4] dates:", dates[1:4].tolist())
print("[1:4] prices:", close[1:4].tolist())
```

實際執行輸出

```
[:3] dates: ['2024-01-02', '2024-01-03', '2024-01-04']
[:3] prices: [185.64, 184.25, 181.91]
[1:4] dates: ['2024-01-03', '2024-01-04', '2024-01-05']
[1:4] prices: [184.25, 181.91, 181.18]
```

兩個切片都有三筆，卻使用不同交易日。若移植MATLAB含右端的1:3，Python不能原樣寫[1:3]：它只會取位置1與2。

討論

在這六筆資料中，close[2:5]選到哪些日期與價格？

原書印201-202 / PDF225-226的部分陣列；Python切片見[2]。

ISSUE 06 / Python附錄

刪一筆之前，如何保留原資料？

重新命名不等於複製；刪除後的新陣列也不等於原陣列已被改掉。

alias=original只讓兩個名稱指向同一物件；separate=original.copy()才另存一份數值。一般NumPy基本切片還可能共用原來的記憶體，想獨立修改時要明寫copy。

原三價185.64、184.25、181.91，np.delete(original,[0,2])另回傳只留184.25的陣列kept。這裡只示範語法，沒有主張應刪除那兩天；市場分析不能為使績效變好任意去樣本。



同由original出發：copy與delete各回傳自己的結果，不是先copy再delete

執行前：執行後original與kept各有幾筆？哪一個變數仍有185.64？

```
import numpy as np
original = np.array([185.64, 184.25, 181.91])
alias = original
separate = original.copy()
kept = np.delete(original, [0, 2])
print("original:", original.tolist())
print("kept middle:", kept.tolist())
print("alias same object:", alias is original)
print("copy shares memory:", np.shares_memory(original, separate))
```

實際執行輸出

```
original: [185.64, 184.25, 181.91]
kept middle: [184.25]
alias same object: True
copy shares memory: False
```

原資料仍有三筆，kept才只剩一筆；alias same object為True，copy shares memory為False。np.delete不是像MATLAB x(indices)=[]那樣直接改原變數。

討論

若只執行np.delete(original,[0])卻沒有接住回傳值，再印original，會剩兩筆嗎？

原書印201-202 / PDF225-226的刪除；NumPy copy/view差異見[1][2]。

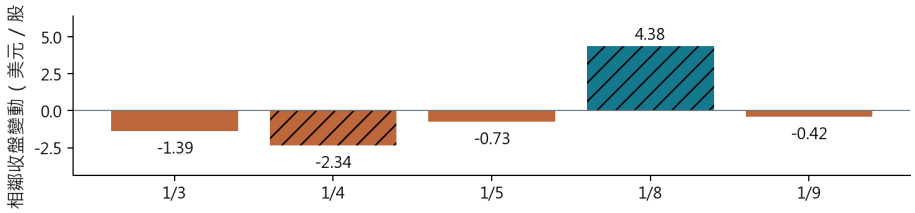
ISSUE 07 / Python附錄

逐日算價格變動，迴圈與陣列運算差在哪？

先看懂一次今天減昨天，再把相同動作一次套用到整段資料。

AAPL 1/3價格變動 = 184.25-185.64 = -1.39美元 / 股。for迴圈的i從1開始，逐筆用close[i]減close[i-1]；range(1,len(close))不包含右端6，因此i依序1至5。

close[1:]為第2至第6價，close[:-1]為第1至第5價；相減讓相鄰日期一一成對。六個價格只能形成五段變動，不能同時把六筆都當成有前收。



2024各段結束日；這是價差，尚未除以前收成為報酬率

執行前：兩種方法會輸出幾段？1/3與1/8變動的正負各是什麼？

```
import numpy as np
close = np.array([185.64, 184.25, 181.91, 181.18, 185.56, 185.14])
loop_change = []
for i in range(1, len(close)):
    loop_change.append(close[i] - close[i - 1])
array_change = close[1:] - close[:-1]
print("loop USD/share:", np.round(loop_change, 2).tolist())
print("array USD/share:", np.round(array_change, 2).tolist())
print("equal:", np.allclose(loop_change, array_change))
```

實際執行輸出

```
loop USD/share: [-1.39, -2.34, -0.73, 4.38, -0.42]
array USD/share: [-1.39, -2.34, -0.73, 4.38, -0.42]
equal: True
```

兩種寫法都得到-1.39、-2.34、-0.73、4.38、-0.42，allclose為True表示在浮點容許差內一致。本例沒有做速度基準測試，不能由行數較少就量化快了幾倍。

討論

若把陣列減法反過來寫close[:-1]-close[1:]，1/3結果變多少？它還代表今天減昨天嗎？

原書印200-201 / PDF224-225的loop及vector operations；本版同一實價作兩實作。

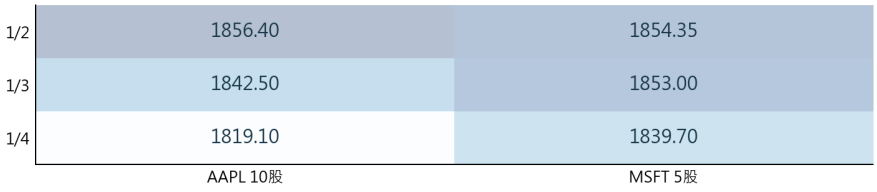
ISSUE 08 / Python附錄

同一組股數，如何對三天兩檔價格估值？

先確定每個軸代表什麼，再讓每檔固定股數對到正確的價格欄。

prices有3個日期、2檔股票，shape=(3,2)；每列依1/2、1/3、1/4，每欄為AAPL、MSFT，皆美元 / 股。另假設固定持AAPL10股、MSFT5股，shares=[10,5]，shape=(2,)。不計費用、股利或交易。

廣播（broadcasting）從最右邊維度配對，這裡2對2，所以每一日期都乘同一組股數。第一天兩欄 = 185.64×10 = 1,856.40美元、370.87×5 = 1,854.35美元。結果仍3×2，但單位由價格變為各檔市值。



2024保存價格×另設固定股數；格內為美元市值，不是報酬率

執行前：prices*shares的shape是否改變？1/2 MSFT欄應乘5還是10？

```
import numpy as np
prices = np.array([[185.64, 370.87], [184.25, 370.60],
                   [181.91, 367.94]])
shares = np.array([10, 5])
values = prices * shares
print("price shape:", prices.shape, "share shape:", shares.shape)
print("stock values USD, AAPL then MSFT:")
print(np.round(values, 2))
```

實際執行輸出

```
price shape: (3, 2) share shape: (2,)
stock values USD, AAPL then MSFT:
[[1856.4  1854.35]
 [1842.5  1853.  ]
 [1819.1  1839.7 ]]
```

每一列都是同一天的兩檔市值；價格沒有改成假行情，也未重新配成固定權重。若改用三個每日係數，不能只看長度3就直接對(3,2)相乘，應先說該係數要沿哪個軸配對。

討論

若只把MSFT改為0股，1/2兩檔市值各是多少？prices是否需要刪掉MSFT欄？

原書印200-203 / PDF224-227陣列操作；廣播規則見[3]，不是原文逐字運算。

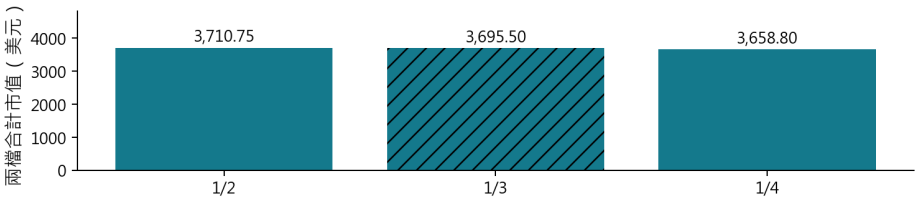
ISSUE 09 / Python附錄

逐元素乘法與矩陣乘法，各算出什麼？

各檔市值與同日總市值是不同的輸出；乘號不能照MATLAB原符號搬過來。

沿前題三天兩檔價格與[10,5]股，prices*shares留下每一天每檔市值；prices@shares則將同一日兩項相加。矩陣乘法的內側長度2相符，輸出為三天各一個總額，shape=(3,)。

1/2總市值 = 185.64×10 + 370.87×5 = 3,710.75美元；1/3為3,695.50，1/4為3,658.80。不含現金的股票總市值不能直接稱完整帳戶權益。



2024固定10股AAPL + 5股MSFT；未加現金 / 費用 / 股利

執行前：兩種乘法結果各有幾個數？@的第一個值是否等於兩個市值相加？

```
import numpy as np
prices = np.array([[185.64, 370.87], [184.25, 370.60],
                   [181.91, 367.94]])
shares = np.array([10, 5])
values = prices * shares
total = prices @ shares
print("elementwise values USD:")
print(np.round(values, 2))
print("matrix total USD:", np.round(total, 2).tolist())
print("sum across stocks agrees:",
      np.allclose(values.sum(axis=1), total))
```

實際執行輸出

```
elementwise values USD:
[[1856.4  1854.35]
 [1842.5  1853.  ]
 [1819.1  1839.7 ]]
matrix total USD: [3710.75, 3695.5, 3658.8]
sum across stocks agrees: True
```

輸出顯示逐元素結果為3×2，@則為長度3。values.sum(axis=1)沿每列的股票欄加總，與@一致。MATLAB一般矩陣乘*在這裡對應Python @；MATLAB逐元素.*對應Python *。

討論

若只想看1/4的AAPL市值而非兩檔合計，應讀values哪個位置，是多少美元？

原書陣列運算之Python補充；*與@區別見[1]，輸入與前題相同。

ISSUE 10 / Python附錄

兩檔股價，要併成幾列幾欄？

相同六個數，日期與股票沿哪個方向排列，會改變後續索引意思。

AAPL三價[185.64,184.25,181.91]，MSFT三價[370.87,370.60,367.94]，皆依2024/1/2、1/3、1/4。column_stack把兩串一維資料作兩欄，形成日期×股票的3×2。

vstack改作兩列，成股票×日期的2×3。concatenate只把兩串一維資料接成長度6，不會保留「這裡有兩檔」的二維形狀。原書以分號上下接合，空白或逗號左右接合；Python應看函式與輸入形狀。



column_stack得到的3×2；全部數字為保存Close，美元 / 股

執行前：三種接法的shape各是什麼？哪一種直接保留每日兩檔的列欄？

```
import numpy as np
aapl = np.array([185.64, 184.25, 181.91])
msft = np.array([370.87, 370.60, 367.94])
by_day = np.column_stack((aapl, msft))
by_stock = np.vstack((aapl, msft))
flat = np.concatenate((aapl, msft))
print("column_stack shape:", by_day.shape, "\n", by_day)
print("vstack shape:", by_stock.shape, "\n", by_stock)
print("concatenate shape:", flat.shape)
```

實際執行輸出

```
column_stack shape: (3, 2)
 [[185.64 370.87]
 [184.25 370.6 ]
 [181.91 367.94]]
vstack shape: (2, 3)
 [[185.64 184.25 181.91]
 [370.87 370.6  367.94]]
concatenate shape: (6,)
```

輸出(3,2)、(2,3)、(6,)說明三種物件不同。若要每日把兩檔市值相加，先確定日期為列，不能對扁平的六數直接sum後說是每日結果。

討論

在by_stock的2×3物件，2024/1/3 MSFT應是哪個位置？

原書印201 / PDF225的concatenation；Python形狀與運算見[1][3]。

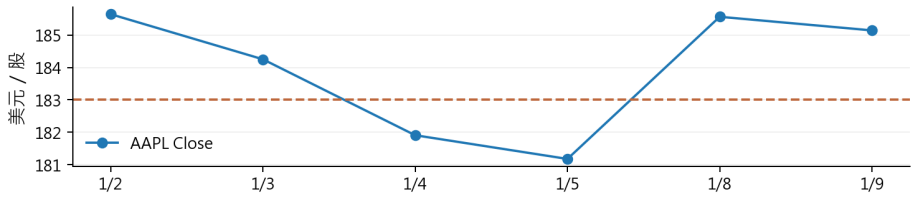
ISSUE 11 / Python附錄

如何只取收盤低於183美元的交易日？

布林條件先標出保留哪筆，再用同一條件選日期與價格。

close<183對六個AAPL價格逐一比較，結果是True或False，這裡稱布林遮罩。183美元 / 股是另設篩選條件，不是買進訊號或市場公認門檻。

1/4的181.91與1/5的181.18小於183，其他四價不符合。dates[mask]與close[mask]使用相同遮罩，才不會把保留下來的價格配錯日期。<是嚴格小於，若恰等於門檻不納入。



2024六交易日與假設183篩選線；線下兩點是1/4、1/5，非策略有效證據

執行前：遮罩有幾個True？取出的第一個日期是不是1/4？

```
import numpy as np
dates = np.array(["2024-01-02", "2024-01-03", "2024-01-04",
                  "2024-01-05", "2024-01-08", "2024-01-09"])
close = np.array([185.64, 184.25, 181.91, 181.18, 185.56, 185.14])
mask = close < 183
print("Close < 183:", mask.tolist())
print("selected dates:", dates[mask].tolist())
print("Close_USD/share:", close[mask].tolist())
```

實際執行輸出

```
Close < 183: [False, False, True, True, False, False]
selected dates: ['2024-01-04', '2024-01-05']
Close_USD/share: [181.91, 181.18]
```

輸出六個布林值，但只選出兩個價格；True不是報酬1，False也不是觀察到0報酬。若用整數[0,1]去選陣列，Python可能解讀為位置0與1，而非布林遮罩。

討論

若門檻改成184.25且仍用<，1/3的184.25會被納入嗎？會保留哪兩天？

原書印201-202 / PDF225-226的logical indexing；Python見[2]。

ISSUE 12 / Python附錄

符合條件的位置，怎樣回到日期？

位置編號不是日期；顯示給人看的第幾筆，也不等於可直接使用的Python索引。

同樣找AAPL Close<183，np.flatnonzero(mask)回傳一維遮罩中True的位置[2,3]，對應第3與第4筆。再用dates[positions]找回1/4、1/5。

MATLAB find對一維例會得到從1起算的[3,4]；移到Python不能直接沿用。多維陣列還有按列或按欄掃描的差異，本頁只處理六筆一維序列，不能將此結論直接套成所有矩陣的find。



2024 AAPL篩選；一組索引同時用在日期與價格

執行前：positions+1是新條件的答案，還是同兩筆資料的人用編號？

```
import numpy as np
dates = np.array(["2024-01-02", "2024-01-03", "2024-01-04",
                  "2024-01-05", "2024-01-08", "2024-01-09"])
close = np.array([185.64, 184.25, 181.91, 181.18, 185.56, 185.14])
positions = np.flatnonzero(close < 183)
print("Python positions:", positions.tolist())
print("human ordinal numbers:", (positions + 1).tolist())
print("dates:", dates[positions].tolist())
```

實際執行輸出

```
Python positions: [2, 3]
human ordinal numbers: [3, 4]
dates: ['2024-01-04', '2024-01-05']
```

Python positions輸出[2,3]；human ordinal numbers加1後[3,4]只是供人閱讀，不應再用它去索引原陣列，否則會錯移一筆交易日資料。

討論

若誤把顯示用的[3,4]拿去選close，會取到哪些日期與價格？

原書印202 / PDF226的find；Python非零位置與索引見[1][2]。

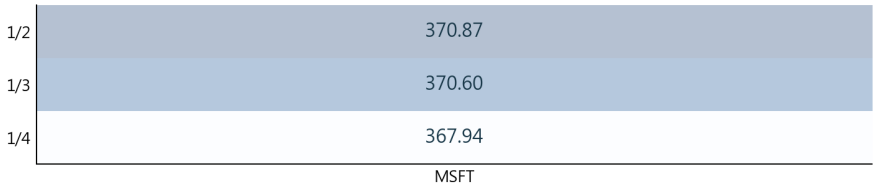
ISSUE 13 / Python附錄

取一天或一檔，為何少了一個維度？

同樣三個數，一維向量與三列一欄，對後續運算有不同形狀。

三日×兩檔prices中，prices[0,:]取第1日兩檔，得到[185.64,370.87]、shape=(2,)。逗號前選日期軸，後選股票軸；冒號表示該軸全部。

prices[:,1]取MSFT三價，得到shape=(3,)；prices[:,1:2]用切片保留股票軸，shape=(3,1)，值仍370.87、370.60、367.94。整數索引通常移除一個軸，切片保留軸，不能只看數值相同。



2024三日單欄Close；格值美元 / 股，shape=(3,1)

執行前：[:,1]與[:,1:2]的值相同，shape是否也相同？

```
import numpy as np
prices = np.array([[185.64, 370.87], [184.25, 370.60],
                   [181.91, 367.94]])
first_day = prices[0, :]
msft_vector = prices[:, 1]
msft_column = prices[:, 1:2]
print("first day:", first_day.tolist(), "shape:", first_day.shape)
print("MSFT vector:", msft_vector.tolist(), msft_vector.shape)
print("MSFT column:", msft_column.tolist(), msft_column.shape)
```

實際執行輸出

```
first day: [185.64, 370.87] shape: (2,)
MSFT vector: [370.87, 370.6, 367.94] (3,)
MSFT column: [[370.87], [370.6], [367.94]] (3, 1)
```

NumPy一維向量沒有獨立的列 / 欄方向，所以一維.T仍是(3,)；要列向量或欄向量需明確建立(1,3)或(3,1)。本頁輸出的兩種MSFT形狀正說明這個差異。

討論

若要保留第1日但仍維持二維，應取prices[0,:]還是prices[0:1,:]？shape是多少？

原書印202 / PDF226選行 / 列；Python降維與切片規則見[1][2]。

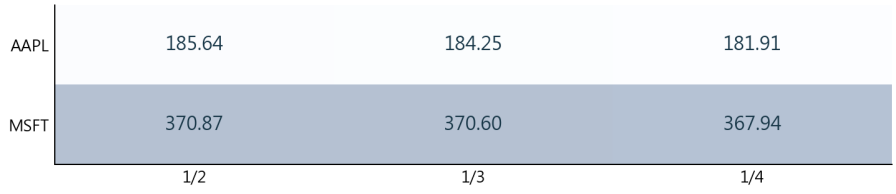
ISSUE 14 / Python附錄

轉置之後，價格代表的日期會改嗎？

轉置只交換資料軸，不會替你重新排序交易日或重算行情。

原prices為1/2、1/3、1/4三列，AAPL、MSFT兩欄。prices.T變成兩檔為列、三日為欄，shape=(2,3)；左上185.64仍是1/2 AAPL，第一列現在是同股三日價格。

np.delete(prices,0,axis=0)另從原prices刪第1日，留下1/3、1/4的2×2；並不是從transposed刪除。原書前後曾沿用x，讀者易失去當前形狀，本版為轉置與刪後物件分別命名。



轉置後仍為保存Close，美元 / 股；同六值換軸，不是改變日期

執行前：轉置後第一列代表股票還是日期？刪第1日後會少哪一軸的一格？

```
import numpy as np
prices = np.array([[185.64, 370.87], [184.25, 370.60],
                   [181.91, 367.94]])
transposed = prices.T
remaining = np.delete(prices, 0, axis=0)
print("transpose shape:", transposed.shape, "\n", transposed)
print("delete first day shape:", remaining.shape, "\n", remaining)
```

實際執行輸出

```
transpose shape: (2, 3)
 [[185.64 184.25 181.91]
 [370.87 370.6  367.94]]
delete first day shape: (2, 2)
 [[184.25 370.6 ]
 [181.91 367.94]]
```

本例都是實數，所以.T符合普通轉置。MATLAB單引號還包含複數共軛，對複數應對照a.conj().T；只需要普通轉置的MATLAB .'才對應.T。不可宣稱兩者在所有型別都等價。

討論

若要從原prices移除MSFT欄，delete應用axis=0還是1？結果shape為何？

原書印202-203 / PDF226-227；轉置、共軛與軸的Python差異見[1]。

ISSUE 15 / Python附錄

reshape換形狀，為何數字位置可能不同？

reshape是按指定順序讀出再填入，不是資料軸的名稱互換。

仍是前題的3×2真價，改成2×3。NumPy預設order="C"先讀完一列：185.64、370.87、184.25…；order="F"先讀完一欄：185.64、184.25、181.91…。兩者讀的是同六個數，但填入新形狀的次序不同。

原書MATLAB的reshape沿欄序，移植時通常需明確order="F"。這也不是.T：F結果第一列185.64、181.91、370.60，與轉置第一列185.64、184.25、181.91不同。不能把reshape後的某欄擅自命名為同一股票三天。



order=F的2×3重排結果；數字源自真價，但新格子不再有原日期×股票語義

執行前：2×3的F結果是否等於prices.T？先比第一列第二格。

```
import numpy as np
prices = np.array([[185.64, 370.87], [184.25, 370.60],
                   [181.91, 367.94]])
c_order = prices.reshape(2, 3, order="C")
f_order = prices.reshape(2, 3, order="F")
print("original, day by stock:\n", prices)
print("reshape C:\n", c_order)
print("reshape F (MATLAB ordering):\n", f_order)
print("transpose equals reshape F:", np.array_equal(prices.T, f_order))
```

實際執行輸出

```
original, day by stock:
 [[185.64 370.87]
 [184.25 370.6 ]
 [181.91 367.94]]
reshape C:
 [[185.64 370.87 184.25]
 [370.6  181.91 367.94]]
reshape F (MATLAB ordering):
 [[185.64 181.91 370.6 ]
 [184.25 370.87 367.94]]
transpose equals reshape F: False
```

輸出transpose equals reshape F為False，直接反駁「只要形狀一樣就是轉置」。squeeze只去掉長度1的軸，也不是轉置或資料對齊；使用前要知道每個軸的意思。

討論

如果問題是把日期×股票改成股票×日期，應用.T還是reshape(2,3,order="F")？為什麼？

原書印203 / PDF227列出的reshape / squeeze；Python順序見[4]。

ISSUE 16 / Python附錄

股票名稱與數字，應放同一個陣列嗎？

能存進去不代表還能按原型別計算；先選能表達資料意義的容器。

原書cell array可在不同格存數值陣列或文字。Python可用清單容納不同物件，用dict記錄欄名與值；不是每個混合物件都應硬塞進同一個數值ndarray。

np.array(["AAPL",185.64])在本例轉成統一字串型別，第二格變成文字"185.64"。dict則用ticker對應文字AAPL、close_USD_per_share對應數字185.64。長度不同的兩份股票名單，可放在外層清單，不必假裝是規則的2×2矩陣。



185.64來自2024/1/2 AAPL Close；各容器承擔不同資料工作

執行前：混合ndarray的第二格仍是浮點數嗎？dict有沒有保留欄名？

```
import numpy as np
mixed = np.array(["AAPL", 185.64])
record = {"ticker": "AAPL", "close_USD_per_share": 185.64}
groups = [["AAPL", "MSFT"], ["AAPL"]]
print("mixed array dtype:", mixed.dtype)
print("second entry is string:", isinstance(mixed[1], str))
print("dictionary:", record)
print("different-length groups:", groups)
```

實際執行輸出

```
mixed array dtype: <U32
second entry is string: True
dictionary: {'ticker': 'AAPL', 'close_USD_per_share': 185.64}
different-length groups: [['AAPL', 'MSFT'], ['AAPL']]
```

dtype的<U32表示此環境得到的Unicode字串儲存，不是股價精度。若只是想把文字"185.64"乘2，會重複文字而不是得到371.28；型別不對就應先查建構步驟。

討論

在record裡，哪個key才能取到可做價格計算的185.64？ticker能拿去算報酬嗎？

原書印203 / PDF227的cell array；Python容器與陣列型別對照見[1]。

ISSUE 17 / Python附錄

np.log如何變成兩天的對數報酬？

先算兩日價格比，再取自然對數，不能把美元股價直接當報酬。

np.log逐元素計算自然對數ln。若要的是對數報酬，輸入應是「今天收盤÷前一交易日收盤」；先除後取log，價格比沒有單位，對數報酬也沒有單位。價格比須為正。

AAPL在2024/1/2、1/3收盤為185.64、184.25美元 / 股。普通報酬 = 184.25÷185.64-1≈-0.748761%；對數報酬 = ln(184.25÷185.64)≈-0.751578%。兩數描述同一價格變動，不是兩筆不同損益。

程式用1/2、1/3、1/4三筆真實收盤配成兩組相鄰價格。兩段對數報酬可相加；還原普通報酬須用np.exp(對數報酬合計)-1，得到181.91÷185.64-1≈-2.009265%，不能直接把對數總和當普通報酬。



AAPL，2024/1/2-1/4；同一兩段Close變動，兩種尺度，非不同交易結果

執行前：兩個價格比都小於1，log後應為正或負？兩段還原後的普通報酬應與哪兩天的端點一致？

```
import numpy as np
close = np.array([185.64, 184.25, 181.91])
ratios = close[1:] / close[:-1]
log_returns = np.log(ratios)
print("price ratios:", np.round(ratios, 8).tolist())
print("log returns, decimal:", np.round(log_returns, 8).tolist())
print("exp(sum) - 1:", round(np.exp(log_returns.sum()) - 1, 8))
```

實際執行輸出

```
price ratios: [0.99251239, 0.98729986]
log returns, decimal: [-0.00751578, -0.01278147]
exp(sum) - 1: -0.02009265
```

1/3的負數代表相對1/2下跌。np.exp(-0.00751578)約為0.99251239；乘185.64會回到184.25。這個轉換方便把多段價格倍率相乘改成對數相加，沒有增加投資收益。

討論

同一筆價格變動若把持有股數由1股改成10股，對數報酬會乘10嗎？請計算這兩種股數的1/2至1/3價格損益。

原附錄PDF227 / 印203，log的逐元素運算；本版延伸為保存AAPL Close的對數報酬。

ISSUE 18 / Python附錄

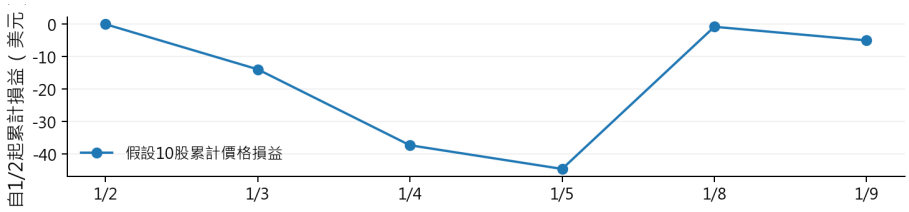
每日損益可以相加，報酬率也能直接加嗎？

用固定股數核對美元損益，再分開報酬的連乘。

sum把元素加總；cumsum保留每一步的累計和。假設2024/1/2收盤起持有AAPL10股至1/9，不交易、不計費用或股利。各日損益＝10×（當日收盤-前收），單位為美元。

1/3為10×(184.25-185.64) = -13.90；1/4再減23.40，累計-37.30。五段依序-13.90、-23.40、-7.30、+43.80、-4.20；加總-5.00，等於10×(185.14-185.64)。

普通報酬每段的分母會變，不能直接相加還原價格。程式把五段小數報酬存為returns；np.prod(1+returns)將五個「期末÷期初」倍率相乘，再減1，約-0.269339%；直接sum(returns)約-0.228928%，不是同一算法。



AAPL真實Close＋固定10股假設；1/5到1/8反彈43.80美元，但尚未填平前跌

執行前：先算1/4的10股累計損益；sum(每日報酬)是否一定等於最後股價÷最初股價-1？

```
import numpy as np
close = np.array([185.64, 184.25, 181.91, 181.18, 185.56, 185.14])
pnl = 10 * np.diff(close)
returns = close[1:] / close[:-1] - 1
compounded = np.prod(1 + returns) - 1
print("daily PnL USD:", np.round(pnl, 2).tolist())
print("cumulative PnL USD:", np.round(np.cumsum(pnl), 2).tolist())
print("sum PnL USD:", round(pnl.sum(), 2))
print("sum returns:", round(returns.sum(), 8))
print("compound:", round(compounded, 8))
```

實際執行輸出

```
daily PnL USD: [-13.9, -23.4, -7.3, 43.8, -4.2]
cumulative PnL USD: [-13.9, -37.3, -44.6, -0.8, -5.0]
sum PnL USD: -5.0
sum returns: -0.00228928
compound: -0.00269339
```

cumsum可看出曾累計虧44.60美元，最後只虧5.00，不能只看最後一個數而忽略中間路徑。上述加總成立於固定10股；若中途換股數，就要每段使用當時實際持有股數。

討論

若同樣全程持有20股，1/5的累計價格損益與1/9的期末價格損益各是多少？1/2至1/9股票報酬是否也加倍？

原附錄PDF227 / 印203，sum、cumsum；np.prod為本版補充的普通報酬連乘。

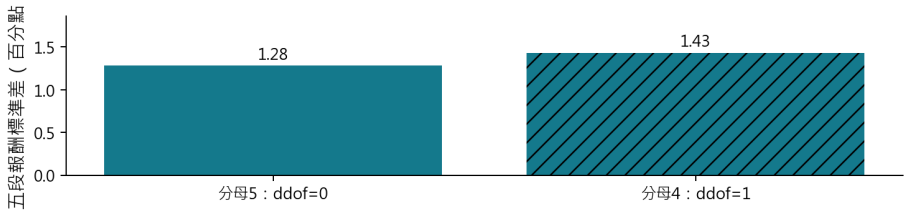
同一組報酬，std為何有兩個答案？

先確認平均、有效筆數與平方偏差的分母，再比較標準差。

AAPL六筆Close產生五段普通報酬，程式存為returns，單位是小數報酬，不是美元。mean為這五數相加除5，約-0.045786%；標準差則看各段偏離這個平均有多遠。

各段減平均後平方，再相加，得到0.0008219602。returns.std(ddof=0)以5為分母：sqrt(0.0008219602÷5)≈0.01282155；ddof=1改除4，約0.01433492。換成百分點為1.282155、1.433492，後者較大是因分母較小。

NumPy的std預設ddof=0；pandas的Series.std與MATLAB的std常用預設為樣本分母。比較前明寫ddof，並固定這五段與相同缺值處理，不把預設不同誤判為行情不同。



AAPL · 2024/1/2-1/9；同一偏差平方和，僅改分母；不是年化波動

執行前：同一平方偏差和，分母5改成4，哪個std會比較大？

```
import numpy as np
close = np.array([185.64, 184.25, 181.91, 181.18, 185.56, 185.14])
returns = close[1:] / close[:-1] - 1
mean = returns.mean()
squares = ((returns - mean) ** 2).sum()
std0 = returns.std(ddof=0)
std1 = returns.std(ddof=1)
print("n:", returns.size, "mean:", round(mean, 8))
print("squared deviations sum:", round(squares, 10))
print("std ddof=0:", round(std0, 8), "ddof=1:", round(std1, 8))
```

實際執行輸出

```
n: 5 mean: -0.00045786
squared deviations sum: 0.0008219602
std ddof=0: 0.01282155 ddof=1: 0.01433492
```

輸出差異來自標準差定義，不是ddof=1多觀察一天。五段資料只描述此子樣本，不能單靠這個數估出未來風險；若後續公式使用樣本標準差，就應沿用ddof=1而不是依數值大小挑選。

討論

不重算五個偏差，請由ddof=0的標準差推得ddof=1應乘的倍率，並說明為何不是直接乘5÷4。

原附錄PDF227 / 印203，mean、std；預設定義及ddof依NumPy、pandas與MathWorks官方文件。

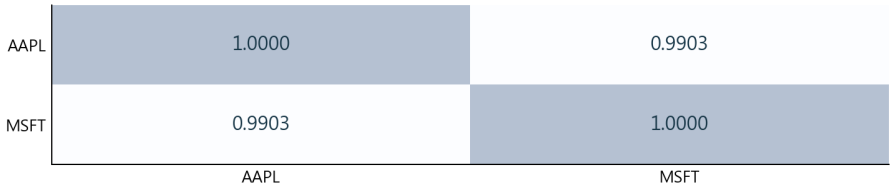
兩檔報酬的相關係數，為何要選rowvar？

先說明一列是一個日期、一欄是一檔股票，才要求程式算股票之間的相關。

AAPL、MSFT各六筆2024/1/2-1/9收盤，形成五個同日期區間的報酬。將兩檔報酬排成shape=(5,2)：五列是五段期間，兩欄依序是AAPL、MSFT。

np.corrcoef(returns,rowvar=False)把每一欄當一個變數，得到2×2矩陣。returns就是下方程式的五段兩檔報酬。對角線1是各檔與自己相關；非對角線約0.990344，是這五對報酬的線性相關。先各扣自己的平均，再用共同偏差衡量是否同向，並除以各自的偏離幅度。

rowvar=True會改把五列當五個變數，只剩每列兩個觀察值，問題已不同。不是任何輸出2×2就正確，還須確認兩欄使用相同日期、相同報酬區間與價格定義。



五對同區間普通報酬，2024/1/2-1/9；對角線自相關，非對角線跨股相關，非預測值

執行前：五列日期、兩欄股票，選rowvar=False後應輸出2×2還是5×5？

```
import numpy as np
prices = np.array([[185.64, 370.87], [184.25, 370.60],
    [181.91, 367.94], [181.18, 367.75],
    [185.56, 374.69], [185.14, 375.79]])
returns = prices[1:] / prices[:-1] - 1
correlation = np.corrcoef(returns, rowvar=False)
print("returns shape:", returns.shape)
print("AAPL/MSFT correlation:\n", np.round(correlation, 6))
print("default rowvar shape:", np.corrcoef(returns).shape)
```

實際執行輸出

```
returns shape: (5, 2)
AAPL/MSFT correlation:
 [[1.       0.990344]
 [0.990344 1.      ]]
default rowvar shape: (5, 5)
```

0.990344說這五對報酬在此小樣本很接近同向變動；它不是99.0344%的獲利率或預測正確率。若用它討論持倉是否重疊，仍應擴展期間並檢查風險，不能當永久相關或因果。

討論

若先把(5,2)報酬矩陣轉置成(2,5)，要保持原來「兩檔股票間相關」的問題，rowvar應改成什麼？結果應是幾乘幾？

原附錄PDF227 / 印203，corrcoef；本版以同日期AAPL、MSFT保存Close演示rowvar。

ISSUE 21 / Python附錄

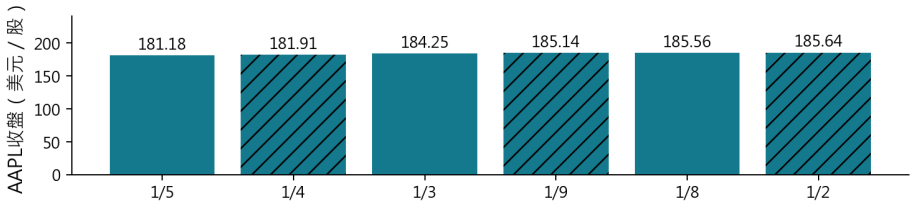
排序價格時，如何不把日期弄丟？

要找最低價及其日期，應排序完整紀錄，而非只排序價格欄。

sort只回傳排序後的值；argsort回傳各值在原陣列的位置。若價格與日期是兩個物件，先取得同一組位置，再同時套在兩者；DataFrame則可用sort_values一次搬動整列。

這六筆AAPL最低是2024/1/5的181.18美元 / 股，原位置3；第二低是1/4的181.91，原位置2。排序位置從[3,2,...]開始，不能把181.18貼回原日期1/2，再宣稱它是1/2價格。

排序可回答最低價發生在哪天，卻會改變時間順序。若要算相鄰交易日報酬，應保留或恢復日期順序；按價格排序後相鄰的兩筆，不一定相隔一個交易日。



2024/1/2-1/9按價格由小至大排序；日期跟著同一筆價格移動，橫軸不再是時間順序

執行前：最低價181.18應和哪一天一起出現在排序後第一筆？原位置是多少？

```
import numpy as np
dates = np.array(["2024-01-02", "2024-01-03", "2024-01-04",
                  "2024-01-05", "2024-01-08", "2024-01-09"])
close = np.array([185.64, 184.25, 181.91, 181.18, 185.56, 185.14])
order = np.argsort(close, kind="stable")
sorted_close = close[order]
print("positions sorted by price:", order.tolist())
for day, price in zip(dates[order], sorted_close):
    print(day, f"{price:.2f}")
```

實際執行輸出

```
positions sorted by price: [3, 2, 1, 5, 4, 0]
2024-01-05 181.18
2024-01-04 181.91
2024-01-03 184.25
2024-01-09 185.14
2024-01-08 185.56
2024-01-02 185.64
```

輸出把1/5與181.18保留為同一筆資料，才能把最低價對回正確日期。argsort的位置沒有日期意義，只有套回原日期陣列後，才知道這個價格是哪個交易日。

討論

已知原日期順序為1/2、1/3、1/4、1/5、1/8、1/9；若只保留排序後的價格而丟掉日期及位置，能由價格本身恢復原來的時間順序嗎？請指出至少一組會被誤當相鄰交易日的排序日期。

原附錄PDF227 / 印203，sort及原列字樣sortrow；正式MATLAB函式為sortrows，Python以argsort或sort_values維持整筆關係。

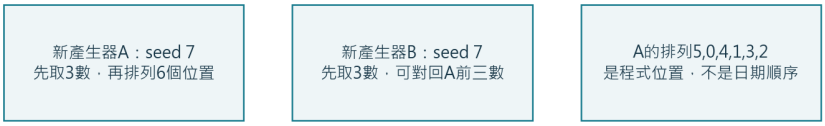
相同亂數種子，為何不保證和MATLAB相同？

把可重現的抽樣操作，與真正的市場觀察分開。

np.random.default_rng(7)建立以種子7初始化的亂數產生器。這裡產生的是0到1之間的三個程式用數字，不是價格、報酬或預測機率；7只是初始化設定，不代表第7個交易日。

同一NumPy設定、相同呼叫順序下，兩個新產生器的前三數同為約0.625095、0.897214、0.775686。隨後permutation(6)把位置0至5重新排列，每個位置恰好出現一次。

若一個產生器已取過三數、另一個才剛建立，再各自抽數時狀態就不同；本程式則讓兩者先做相同操作。MATLAB與Python也可能採不同產生器或抽樣演算法，不能只寫同一種子7就聲稱跨語言必相同。



A、B是分開建立的產生器；相同初始設定與取數操作可核對，不是三個連續處理階段

執行前：同種子、同環境、同呼叫順序的前三數是否相同？permutation(6)會不會重複某個位置？

```
import numpy as np
first = np.random.default_rng(7)
second = np.random.default_rng(7)
random_values = first.random(3)
repeated = second.random(3)
shuffled_positions = first.permutation(6)
print("seed 7 raw random numbers:", np.round(random_values, 6).tolist())
print("same NumPy setup repeats:", np.array_equal(random_values, repeated))
print("permutation, not time order:", shuffled_positions.tolist())
```

實際執行輸出

```
seed 7 raw random numbers: [0.625095, 0.897214, 0.775686]
same NumPy setup repeats: True
permutation, not time order: [5, 0, 4, 1, 3, 2]
```

排列[5,0,4,1,3,2]可用於明示的抽樣練習；若依此打亂金融時間序列再切訓練 / 測試，未來日期可能跑到訓練中，已不是原來的時間順序評估。可重現不等於研究設計合理。

討論

將此排列套到依序1/2、1/3、1/4、1/5、1/8、1/9六個日期，第一個與最後一個日期會是哪天？它們是否仍表示樣本起點與終點？

原附錄PDF227 / 印203，rand、randperm；NumPy default_rng的種子與產生器語義依官方文件。

ISSUE 23 / Python附錄

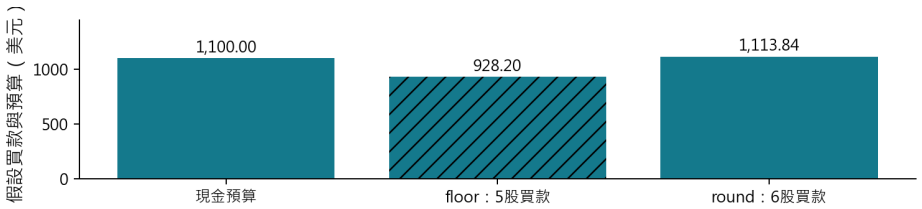
可買幾股，round與floor為何不同？

整股購買要檢查總買款不超預算，不能只取最近整數。

【購買數量假設】可用現金1,100美元，以AAPL2024/1/2收盤185.64美元 / 股作估價，不計費用、不借款。允許碎股時，1,100÷185.64≈5.925447股；不允許碎股時，最多買floor(5.925447) = 5股。

floor往不大於輸入的整數取值，5股買款928.20，餘171.80美元。round取最近整數得到6股，但6×185.64 = 1,113.84，反而超預算13.84。捨入規則與可負擔性是兩個問題。

np.round遇到正好在兩整數中點時，採接近的偶數，例如純數字2.5→2、3.5→4；這不是一般口語「逢5必進」。ceil往不小於輸入的整數取值，這裡也會給6，仍要檢查資金。



AAPL 2024/1/2 Close為估價基準；整股 / 碎股及不收費是教學假設，非成交保證

執行前：1,100÷185.64略小於6，round得到的6股是否真的付得起？

```
import numpy as np
cash, close_jan2 = 1100.0, 185.64
fractional = cash / close_jan2
whole = int(np.floor(fractional))
nearest = int(np.round(fractional))
print("fractional shares:", round(fractional, 6))
print("floor shares:", whole, "cash left USD:",
      round(cash - whole * close_jan2, 2))
print("rounded shares:", nearest, "cost USD:",
      round(nearest * close_jan2, 2))
print("round ties:", np.round([2.5, 3.5]).tolist())
```

實際執行輸出

```
fractional shares: 5.925447
floor shares: 5 cash left USD: 171.8
rounded shares: 6 cost USD: 1113.84
round ties: [2.0, 4.0]
```

floor適用於本題正價格、整股、無費用且不借款的預算限制；若有最低費、每股費或保留現金，須先扣掉相關需求再判斷股數，而非一律套同一個除式。

討論

同價格與1,100美元預算，若另假設每次買進固定收10美元，最多可買幾整股，付完費後剩多少現金？

原附錄PDF227 / 印203，round、floor、ceil、fix；本版用保存AAPL Close與明示預算說明。

ISSUE 24 / Python附錄

兩份股票名單，交集會保留重複與順序嗎？

先決定要的是不同股票的集合，還是原資料中符合條件的每一筆。

本題只有名稱選取，沒有持股建議。left依序為MSFT、AAPL、AAPL；right為AAPL、TSLA。np.intersect1d找兩邊共同的不同值，預設排序並去除重複，所以只有一個AAPL。

np.isin(left,right)則逐筆回答left的名字是否在right，得到False、True、True；left[mask]會保留原順序及兩筆AAPL。聯集union1d是AAPL、MSFT、TSLA；setdiff1d(left,right)是只有左邊有的MSFT。

mask.any()表示至少一筆符合，為True；mask.all()表示每筆都符合，為False。兩份名單有交集，不代表兩份資料的每一筆或順序相同；價格表合併仍需按股票代號與日期對回。



由同一組left / right分別比較兩種取法，不是先取交集再做isin；重複列不是新增股票

執行前：交集只保留一個不同名稱；isin後的left會保留幾筆AAPL？

```
import numpy as np
left = np.array(["MSFT", "AAPL", "AAPL"])
right = np.array(["AAPL", "TSLA"])
common = np.intersect1d(left, right)
mask = np.isin(left, right)
print("unique sorted intersection:", common.tolist())
print("left-order matched names:", left[mask].tolist())
print("union:", np.union1d(left, right).tolist())
print("left minus right:", np.setdiff1d(left, right).tolist())
print("any:", mask.any(), "all:", mask.all())
```

實際執行輸出

```
unique sorted intersection: ['AAPL']
left-order matched names: ['AAPL', 'AAPL']
union: ['AAPL', 'MSFT', 'TSLA']
left minus right: ['MSFT']
any: True all: False
```

若重複AAPL原本來自兩個不同帳戶，直接去重會丟掉一筆帳戶紀錄；若只是要建立不同股票清單，保留重複又會重複計數。先確認每筆代表什麼，再選集合或逐筆篩選。

討論

若要知道left三筆中「不在right」的是哪幾筆，應怎樣改mask？改完any與all各為何？

原附錄PDF227 / 印203，setdiff、union、intersect、ismember、unique、any、all；本版對應NumPy集合及逐筆判斷。

ISSUE 25 / Python附錄

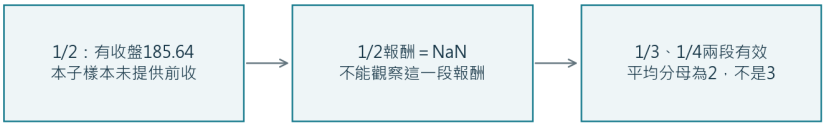
第一筆報酬是NaN，能直接填0嗎？

分開「沒有前一筆價格」與「已觀察到價格不變」。

本題只提供AAPL2024/1/2、1/3、1/4三筆Close：185.64、184.25、181.91美元 / 股。pct_change(fill_method=None)以前一筆為分母；子樣本第一天沒有提供前收，故1/2報酬為NaN，不是市場當天報酬0%。

其餘兩段約-0.00748761、-0.01270014。平均只用這兩個有效數：(-0.00748761-0.01270014)÷2≈-0.01009387。若把第一筆填0再平均，分母變3，約-0.00672925；加入了一個沒有觀察依據的零報酬。

np.isnan辨認NaN；np.isfinite只接受有限數值，NaN及正負無限大都為False。兩者回傳的是布林判斷，不是報酬或價格。先說缺值來源，再選保留、補資料或排除的範圍。



AAPL 2024/1/2-1/4；自然形成的起始NaN，並非人為刪價或把價格填0

執行前：三筆收盤能計幾段報酬？填入一個0後，平均的分母會變成多少？

```
import numpy as np
import pandas as pd
dates = ["2024-01-02", "2024-01-03", "2024-01-04"]
close = pd.Series([185.64, 184.25, 181.91], index=dates)
returns = close.pct_change(fill_method=None)
print(returns.rename("return_decimal").to_string(float_format="%.8f"))
print("isnan:", np.isnan(returns).tolist())
print("finite:", np.isfinite(returns).tolist())
print("valid mean:", round(returns.mean(), 8))
print("zero-filled mean:", round(returns.fillna(0).mean(), 8))
```

實際執行輸出

```
2024-01-02           NaN
2024-01-03   -0.00748761
2024-01-04   -0.01270014
isnan: [True, False, False]
finite: [False, True, True]
valid mean: -0.01009387
zero-filled mean: -0.00672925
```

輸出中的有效平均約-1.009387%，補0後約-0.672925%；差額來自新增零值及分母，而非股票比較少跌。若能補齊同一價格定義的前一交易日收盤，才能計算第一段，不應用0掩蓋缺少的期間。

討論

若下一位同學拿到同樣三筆Close，卻說「有效報酬有三筆，因為三個價格都是有限值」，請指出他混淆了哪兩種物件。

原附錄PDF227 / 印203，isnan、isfinite、islogical；本版以自然起始NaN及pandas報酬計算說明。

ISSUE 26 / Python附錄

從欄名取股票代號，為何不使用eval？

把欄名當文字查找與轉換，不把收到的文字當程式執行。

資料欄名AAPL_Close、MSFT_Close、AAPL_Volume都是文字。若目的只是取收盤欄，先用endswith("_Close")篩選，再用re.sub把字尾_Close去掉，就得到股票代號AAPL、MSFT。這裡沒有計算價格。

re是Python的正規表示式模組；r"_Close$"的$限定字尾，避免把欄名中間相同片段也刪掉。re.fullmatch(r"[A-Z]+_Close",s)要求整個欄名符合「一個以上大寫英文字母＋_Close」，不是只找到其中一小段。

eval會把字串當Python運算式執行，不能拿來處理來源不可信的欄名。用字串方法、正規表示式或字典依名稱取值，就能完成本題，不需要讓文字取得執行程式的能力。



欄名文字的篩選與清理；輸出代號沒有美元單位，也不是股票價格或交易訊號

執行前：AAPL_Volume是否應出現在Close欄結果？整個欄名模式對它應回True還是False？

```
import re
columns = ["AAPL_Close", "MSFT_Close", "AAPL_Volume"]
close_columns = [s for s in columns if s.endswith("_Close")]
tickers = [re.sub(r"_Close$", "", s) for s in close_columns]
matches = [bool(re.fullmatch(r"[A-Z]+_Close", s)) for s in columns]
print("Close columns:", close_columns)
print("ticker text:", tickers)
print("whole-name pattern matches:", matches)
```

實際執行輸出

```
Close columns: ['AAPL_Close', 'MSFT_Close']
ticker text: ['AAPL', 'MSFT']
whole-name pattern matches: [True, True, False]
```

程式只保留兩個Close欄；模式結果True、True、False能排除AAPL_Volume。這個模式是本題指定格式，不涵蓋所有真實市場代號，例如含點號者需要另定格式，不能誤說它不是合法股票。

討論

若新增BRK.B_Close，它會通過endswith("_Close")嗎？會通過本題的[A-Z]+_Close完整模式嗎？這代表應刪掉資料嗎？

原附錄PDF224 / 印200的文字處理及PDF227 / 印203的strmatch、regexp、regexprep、eval；本版以安全文字操作替代eval。

ISSUE 27 / Python附錄

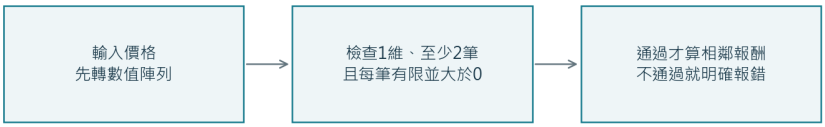
把報酬計算寫成函式，怎樣檢查輸入？

讓重複計算有清楚的輸入要求，資料不足時直接說明原因。

def定義可重複呼叫的函式。simple_returns接收一維、至少兩筆、有限且大於0的價格，回傳相鄰普通報酬；np.asarray把可接受的輸入轉成數值陣列，並不是自動確認資料日期或價格定義正確。

先檢查維度及筆數，再檢查每個價格，最後才除法。185.64、184.25、181.91會產生兩筆報酬，第一筆184.25÷185.64-1≈-0.00748761。只有185.64一筆時沒有前後組合，raise ValueError指出缺少歷史，不回傳虛構0%。

try執行可能失敗的呼叫，except ValueError只處理已指定的輸入錯誤。捕捉後把原因顯示出來；不能把所有錯誤默默改成零報酬，讓後續平均或回測繼續使用。



函式的輸入限制與運算順序；例子是保存AAPL Close，未檢查交易時點或可成交性

執行前：三筆價格應回傳幾筆報酬？只有一筆價格時應回傳0，還是指出資料不足？

```
import numpy as np
def simple_returns(close):
    close = np.asarray(close, dtype=float)
    if close.ndim != 1 or close.size < 2:
        raise ValueError("need at least two prices in one dimension")
    if not np.all(np.isfinite(close) & (close > 0)):
        raise ValueError("prices must be finite and positive")
    return close[1:] / close[:-1] - 1
returns = simple_returns([185.64, 184.25, 181.91])
print("returns:", np.round(returns, 8).tolist())
try:
    simple_returns([185.64])
except ValueError as error:
    print("incomplete input:", error)
```

實際執行輸出

```
returns: [-0.00748761, -0.01270014]
incomplete input: need at least two prices in one dimension
```

正常輸出兩筆負報酬；只有一筆價格時顯示need at least two prices in one dimension。這讓讀者知道缺的是前後價格配對，而不是把函式跑完誤稱資料可用。

討論

若把同一組三筆真實價格排成shape=(3,1)再傳入此函式，為何仍應拒絕？應先做哪一項核對，才決定是否轉成一維？

原附錄PDF227 / 印203的try、catch及PDF228 / 印204自建工具函式；本版用Python def、raise與except。

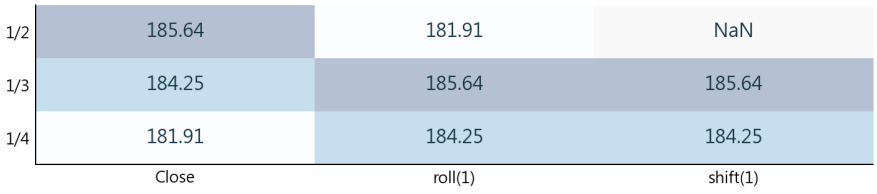
ISSUE 28 / Python附錄

為何np.roll不能直接當前一天價格？

區分循環搬動元素，與把前一筆價格對到今天。

np.roll(x,1)把元素往後搬一格，最後一筆會繞回第一格。AAPL這份2024/1/2-1/4子樣本為185.64、184.25、181.91；roll後為181.91、185.64、184.25。第一格的181.91來自1/4，不是1/2之前的價格。

Series.shift(1)則保留原日期，把前一筆值移到今天。1/2沒有本題提供的前收，留NaN；1/3對到1/2的185.64，1/4對到1/3的184.25。這才符合相鄰收盤報酬的分母。



AAPL · 2024/1/2-1/4 · 美元 / 股；roll首格取到未來1/4 · shift首格正確保留缺值

執行前：roll的第一格會來自哪一天？shift首格是否應有已知前收？

```
import numpy as np
import pandas as pd
dates = ["2024-01-02", "2024-01-03", "2024-01-04"]
close = pd.Series([185.64, 184.25, 181.91], index=dates)
rolled = np.roll(close.to_numpy(), 1)
previous = close.shift(1)
result = pd.DataFrame({"Close": close, "roll": rolled, "shift": previous})
print(result.to_string(float_format="%.2f"))
```

實際執行輸出

```
            Close   roll  shift
2024-01-02 185.64 181.91    NaN
2024-01-03 184.25 185.64 185.64
2024-01-04 181.91 184.25 184.25
```

表格後兩筆看似相同，問題藏在第一筆。如果用roll當前收，1/2就偷用了1/4價格；不能因程式沒有NaN或沒有報錯，就說每筆報酬都有合理日期。

討論

若要算「隔兩個交易日」的價格報酬，改用shift(2)，本題哪一天首次可計算？請列出那一段的兩個價格及報酬。

原附錄PDF227 / 印203，circshift；本版對照NumPy循環roll與pandas非循環shift。

ISSUE 29 / Python附錄

按日期取價，loc與iloc各選哪一筆？

能用股票代號及日期指明的資料，不必靠記住它碰巧排第幾筆。

DataFrame的列標籤是以pd.to_datetime建立的日期，欄名是AAPL、MSFT。loc按標籤選取；iloc按從0開始的位置選取。2024/1/3的MSFT收盤370.60美元 / 股，可用loc["2024-01-03","MSFT"]取到。

在本題依日期排序、欄序AAPL後MSFT的表中，iloc[1,1]也取到370.60；這只因指定位置剛好對應同一日期及股票，不代表日期永遠是第1位置。新增、刪除、排序或換欄序後，位置可能改變。

loc以日期切片時通常包含兩個端點，1/2:1/3選兩天；iloc[0:1]沿用Python右端不包含的規則，只選第0位置一筆。標籤範圍與位置範圍不能看起來相似就互換。

	AAPL	MSFT
1/2	185.64	370.87
1/3	184.25	370.60
1/4	181.91	367.94

保存Close，2024年，美元 / 股；loc指定1/3及MSFT，iloc[1,1]依此表位置才碰巧相同

執行前：本表loc與iloc能否取到同一個370.60？兩種切片各會留下幾天？

```
import pandas as pd
dates = pd.to_datetime(["2024-01-02", "2024-01-03", "2024-01-04"])
prices = pd.DataFrame({"AAPL": [185.64, 184.25, 181.91],
                       "MSFT": [370.87, 370.60, 367.94]}, index=dates)
label_value = prices.loc["2024-01-03", "MSFT"]
position_value = prices.iloc[1, 1]
print("loc 2024-01-03 MSFT USD/share:", label_value)
print("iloc [1,1] USD/share:", position_value)
print("inclusive loc dates:",
      prices.loc["2024-01-02":"2024-01-03"].shape[0])
print("exclusive iloc [0:1] rows:", prices.iloc[0:1].shape[0])
```

實際執行輸出

```
loc 2024-01-03 MSFT USD/share: 370.6
iloc [1,1] USD/share: 370.6
inclusive loc dates: 2
exclusive iloc [0:1] rows: 1
```

兩個取值皆為370.6，顯示相同價格可以有兩種明確選法；輸出2與1是各自切片的筆數，不是報酬，也不代表兩天有兩筆交易。按日期計算時，優先讓選取條件說出真正需要的日期。

討論

若只把兩欄順序改成MSFT、AAPL，資料與日期均不變，原loc查詢及原iloc[1,1]各取到多少？哪一個仍回答「1/3的MSFT」？

原附錄PDF227-228 / 印203-204，table、timetable及按日期 / 代號取值；本版對照pandas loc、iloc。

ISSUE 30 / Python附錄

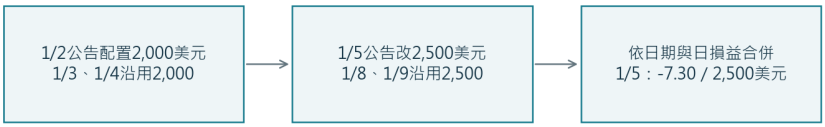
損益與資金配置，合併時怎樣保留正確日期？

按日期合併兩種紀錄，並說清楚哪個設定會持續生效。

AAPL2024/1/2-1/9假設持有固定10股，不交易或計費，日損益＝10×收盤差。另一份是假設的研究資金配置：1/2公告2,000美元、1/5起改2,500，未再次公告前沿用。配置不是股票市值、帳戶權益，也不是當日獲利或入金紀錄。

events.reindex(dates,method="ffill")把配置對到六個交易日，空白日沿用最近已公告值：1/3、1/4仍2,000，1/5、1/8、1/9為2,500。這只因本題明定配置持續有效，不是把缺失的市場價格猜出來。

join(...,how="left")保留左邊六個日期，再把配置併欄。1/5會同時看到日損益-7.30、配置2,500；兩者不可相加叫作獲利。1/2日損益仍NaN，因為本子樣本未提供前收，不能順手一起填0。



股數及配置生效規則為教學假設；價格為真實Close，未補價格或把配置變化算損益

執行前：1/4與1/5各應沿用哪個配置金額？1/2的損益NaN會因合併而自動變0嗎？

```
import pandas as pd
dates = pd.to_datetime(["2024-01-02", "2024-01-03", "2024-01-04",
                        "2024-01-05", "2024-01-08", "2024-01-09"])
close = pd.Series([185.64,184.25,181.91,181.18,185.56,185.14], index=dates)
pnl = (10 * close.diff()).rename("PnL_USD")
event_dates = pd.to_datetime(["2024-01-02", "2024-01-05"])
events = pd.Series([2000., 2500.], index=event_dates)
allocation = events.reindex(dates, method="ffill").rename("allocation_USD")
combined = pnl.to_frame().join(allocation, how="left")
print(combined.to_string(float_format="%.2f"))
```

實際執行輸出

```
            PnL_USD  allocation_USD
2024-01-02      NaN         2000.00
2024-01-03   -13.90         2000.00
2024-01-04   -23.40         2000.00
2024-01-05    -7.30         2500.00
2024-01-08    43.80         2500.00
2024-01-09    -4.20         2500.00
```

合併結果可回答「那一天計得多少損益、當天指定多少配置」，不能單靠兩欄還原真實帳戶。若要把配置當報酬分母，仍須另定生效時點、資金流與實際持股規則。

討論

若另公告1/8起配置改為2,200美元，其他條件不變，1/5、1/8、1/9各沿用多少配置？1/8的43.80美元日損益應否因此改寫？

原附錄PDF228 / 印204，synchronize按日期合併；本版用pandas reindex、ffill及join，明示配置生效條件。

ISSUE 31 / Python附錄

每日損益轉月損益，為何不能全用sum？

月內美元損益可以加，價格報酬仍要連乘，還要辨認資料是否涵蓋整月。

本題仍只有2024/1/2-1/9六筆AAPL Close，不是完整一月。resample("ME")按曆月分組，標籤放月末；即使結果標成1/31，也不會憑空補出1/10-1/31未提供的價格或損益。

固定10股、無交易與費用，五段美元損益相加：-13.90-23.40-7.30 + 43.80-4.20 = -5.00。sum(min_count=1)要求分組至少有一個有效值；全部缺值時保留缺值，不冒充觀察到0美元。

五段普通報酬先各加1再prod，最後減1，約-0.00269339。端點也可核對：185.14÷185.64-1約-0.269339%。這是1/2收盤至1/9收盤的子樣本報酬，不能標成整月績效。



同一子樣本，不同計量單位使用不同聚合；月末標籤不等於已取得完整月資料

執行前：五段固定10股損益加總應等於哪兩個收盤的價差乘10？

```
import pandas as pd
dates = pd.to_datetime(["2024-01-02", "2024-01-03", "2024-01-04",
                        "2024-01-05", "2024-01-08", "2024-01-09"])
close = pd.Series([185.64,184.25,181.91,181.18,185.56,185.14], index=dates)
pnl = 10 * close.diff()
returns = close.pct_change(fill_method=None)
month_pnl = pnl.resample("ME").sum(min_count=1)
month_return = (1 + returns).resample("ME").prod(min_count=1) - 1
print("Jan 2-9 sample PnL USD:", round(month_pnl.iloc[0], 2))
print("sample compounded return:", round(month_return.iloc[0], 8))
print("endpoint checks:", round(10*(close.iloc[-1]-close.iloc[0]), 2),
      round(close.iloc[-1]/close.iloc[0]-1, 8))
```

實際執行輸出

```
Jan 2-9 sample PnL USD: -5.0
sample compounded return: -0.00269339
endpoint checks: -5.0 -0.00269339
```

兩個端點核對分別得到-5美元與-0.269339%，支持這份子樣本的加總與連乘。若要回報完整一月，應先補齊同價格基準的月末資料，並說清前一月末至本月末的起訖定義。

討論

若只保留1/2至1/5四筆收盤，固定10股且其餘條件不變，子樣本美元損益與普通報酬各是多少？能把它標成完整1月報酬嗎？

原附錄PDF228 / 印204，retime日損益轉月損益；本版對照pandas resample及不同數量的聚合。

ISSUE 32 / Python附錄

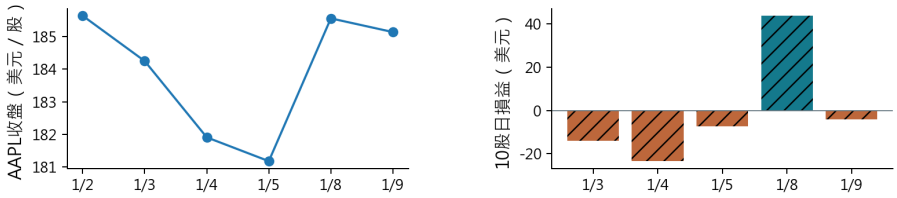
股價與美元損益，圖上要怎樣分開？

兩張圖分別回答每股值多少、固定股數每天增減多少美元。

左圖畫2024/1/2-1/9 AAPL Close，縱軸為美元 / 股，每個點是當天一股收盤值。右圖畫固定持有10股的各段價格損益，縱軸為美元；柱的日期是該段終點，不是買進訊號。

1/8收盤185.56高於1/5的181.18，故右圖1/8柱為10×(185.56-181.18) = +43.80美元。雖然它是最大正柱，當時股價仍比1/2的185.64低0.08，累計10股仍少0.80美元；日損益與累計損益不能混用。

Matplotlib的plot連接時間路徑，bar比較每天正負金額；hist分箱、scatter比較成對數值，各回答不同問題。程式將兩張圖存成執行目錄的issue32_aapl.png；執行後開啟該圖即可看結果，不必改notebook後端。



2024年真實Close + 固定10股、不交易及無費用假設；左右縱軸單位不同，不重疊共用刻度

執行前：1/8應在哪一張圖顯示185.56、哪一張圖顯示43.80？兩者單位各是什麼？

```
import matplotlib.pyplot as plt
import numpy as np
dates = ["01-02","01-03","01-04","01-05","01-08","01-09"]
close = np.array([185.64,184.25,181.91,181.18,185.56,185.14])
pnl = 10 * np.diff(close)
fig, (price_ax, pnl_ax) = plt.subplots(1, 2, figsize=(9, 3))
price_ax.plot(dates, close, "o-")
pnl_ax.bar(dates[1:], pnl)
price_ax.set(xlabel="2024 date", ylabel="Close USD/share")
pnl_ax.set(xlabel="2024 ending date", ylabel="daily PnL USD")
fig.tight_layout()
fig.savefig("issue32_aapl.png", dpi=150)
axis_count = len(fig.axes)
axis_labels = [ax.get_ylabel() for ax in fig.axes]
print(axis_count, "axes;", " | ".join(axis_labels))
print("saved: issue32_aapl.png")
plt.close(fig)
```

實際執行輸出

```
2 axes; Close USD/share | daily PnL USD
saved: issue32_aapl.png
```

程式由fig.axes實際數到2個座標軸，並讀出兩軸不同單位；PNG可對回圖中日期與價格。右圖1/8單段獲利不表示從任意起日投入都賺，也不能把未模擬的下單與成交畫成已實現報酬。

討論

若仍固定10股、只觀察1/8至1/9，右圖應是賺或賠多少？為何不能把這一根柱直接當成從1/2起的累計損益？

原附錄PDF224 / 印200圖形功能及PDF227 / 印203的plot、hist、bar、scatter；本版以Matplotlib與保存AAPL價格作圖。

原書語法移到Python，哪些不能照抄？

正文先把每個操作算清楚；此頁集中說明跨平台語義，避免把不同物件或預設當作同一件事。

顯示與物件大小

MATLAB的%註解換成Python的#。Python腳本用print明確顯示結果；notebook可能自動顯示最後一個運算式。MATLAB可用索引賦值延長陣列，NumPy越界賦值會報錯；(3,)也不是MATLAB的1×3列向量。見第1-4、13題。

先決定在哪個軸運算

對本版日期×股票矩陣，axis=0沿日期逐欄彙整；axis=1沿股票逐日彙整。NumPy的sum、mean、std不寫axis會縮成單值，不能默認和MATLAB逐欄結果相同。第19題另以ddof選分母；MATLAB std(x,1)的1選分母N，不是指定第1維。[1][5][21]

多維位置與轉置

第12題只做一維find。多維時，np.nonzero回傳各軸位置組；flatnonzero按列攤平後給位置，而MATLAB線性索引按欄、從1起算。先對回原日期與股票，不能只把索引減1。第14-15題區分轉置與F / C重排；不熟時先畫出每格原值。[1][2][4]

取單值還是保留子表

原書table可有一般列標籤，timetable用時間索引。第29題loc[日期,代號]取單一價格，loc[[日期],[代號]]則保留一列一欄的DataFrame；兩者不是同一型別。原書第1562列、第244欄若改位置索引，對應iloc[1561,243]，前提是表的排列完全未變。[14]

日期合併不是自動修好資料

MATLAB synchronize不指定新時間時取兩邊日期聯集，未配到處留缺值；pandas join預設left。本版第30題明寫how="left"；若改outer，才保留兩邊全部日期。配置事件依已公告且持續生效的假設前填，不能一起前填所有損益或行情。[11][23]

月份標籤與檔案重用

MATLAB retime的彙整預設用區間左端標記；第31題pandas ME標月末，MS標月初。標籤不會補出缺少的行情。自建函式可存成.py模組再import；檔名不要取numpy.py、pandas.py或random.py，以免擋住要匯入的套件。這些是移植查找方向，不宣稱本版執行了原MATLAB程式。[12][24]

原書50項函式：數值彙整與形狀

原名保留便於查原書；下表是查找方向，不宣稱所有預設、型別、缺值或版本等價。「相關題」連結概念，不保證該題示範了此API；只有正文程式區實際顯示的寫法才在本版執行。

原MATLAB名	Python方向	用途與必要差異	相關題
sum	np.sum(a,axis=...)	沿哪個軸加總要明寫；美元加總不等於報酬複利。	18
cumsum	np.cumsum(a,axis=...)	保留各步累計；未寫axis可能攤平。	18
diag	np.diag	一維輸入建立對角矩陣；二維輸入取對角線。	20
max	np.max / np.maximum	全體 / 沿軸最大值與逐對比較不同；np.max遇NaN回NaN，先定缺值處理。	25
min	np.min / np.minimum	全體 / 沿軸最小值與逐對比較不同；np.min遇NaN回NaN，先定缺值處理。	25
mean	np.mean / Series.mean	軸與有效筆數須相同；pandas通常略過缺值。	19、25
std	np.std(...,ddof=1)	對照MATLAB常見樣本分母；NumPy預設ddof=0。	19
corrcoef	np.corrcoef	日期為列、變數為欄時用rowvar=False。	20
repmat	np.tile / broadcasting	tile實際複製排列；廣播是運算時配形狀，別混叫同一物件。	8
reshape	a.reshape(...,order="F")	要相容MATLAB欄序時指定F；不是轉置。	15
squeeze	np.squeeze	去長度1的軸；可能變1維或0維，不保留MATLAB最少2維慣例。	13、15

原附錄印203 / PDF227短清單；原拼字sortrow在第二表註明。API見文末[1]-[24]；需要哪個軸、分母與排序必須由當題資料決定。

原書50項函式：排序、大小與整數運算

原名保留便於查原書；下表是查找方向，不宣稱所有預設、型別、缺值或版本等價。「相關題」連結概念，不保證該題示範了此API；只有正文程式區實際顯示的寫法才在本版執行。

原MATLAB名	Python方向	用途與必要差異	相關題
sort	np.sort(a,axis=...)	依指定軸排序；不要把每一價格欄各排完還保留原日期。	21
sortrow	argsort索引 / sort_values	原書拼法應為MATLAB sortrows；按指定欄排序整列。	21
rand	rng.random	由Generator產生；相同數字種子不保證跨語言相同序列。	22
size	a.shape / a.size	shape是各維長度；size是總元素數，兩者不可互代。	13
length	依物件選len或max(shape)	MATLAB矩陣length是最大維長；len(a)通常只數第一軸。	13
eigs	scipy.sparse.linalg.eigs	求指定部分特徵對；k、which等選擇須對齊，不是全部eig。	見[15]
fix	np.trunc / np.fix	往0取整，負數時不同於floor。	23
round	np.round / round	NumPy精確中點取偶數；勿默認與MATLAB規則相同。	23
floor	np.floor	向負無窮取整；正預算買整股時不向上超支。	23
ceil	np.ceil	向正無窮取整；不是所有整股情境都適用。	23
mod	np.mod	檢查除數符號與0；不是向0截斷後的餘數定義。	23
factorial	math.factorial	標量非負整數；需要陣列時可逐項呼叫或另核SciPy介面。	純計數工具

原附錄印203 / PDF227短清單；原拼字sortrow在第二表註明。API見文末[1]-[24]；需要哪個軸、分母與排序必須由當題資料決定。

原書50項函式：集合、布林與名稱查找

原名保留便於查原書；下表是查找方向，不宣稱所有預設、型別、缺值或版本等價。「相關題」連結概念，不保證該題示範了此API；只有正文程式區實際顯示的寫法才在本版執行。

原MATLAB名	Python方向	用途與必要差異	相關題
setdiff	np.setdiff1d	不同元素的差集，通常排序並去重；不等於保留原每一列。	24
union	np.union1d	兩組不同名稱的聯集；不能據此猜價格表順序。	24
intersect	np.intersect1d	交集通常去重；不要拿結果替代原重複帳戶紀錄。	24
ismember	np.isin	逐元素布林值；若需MATLAB第二個位置輸出，另做有重複規則的索引。	24
unique	np.unique	取得不同值；是否排序、保留首次位置要分開指定 / 處理。	24
any	np.any / mask.any	至少一個True；注意軸與空集合語義。	24
all	np.all / mask.all	全部True；不代表資料沒有缺價或時間正確。	24、27
eval	不以eval處理資料文字	字典取值、明確函式或字串解析；不執行不可信輸入。	26
eye	np.eye	建立單位矩陣，需指定階數；不是現金配置或價格表。	矩陣工具
ones	np.ones	建立全1的指定形狀；值意義由問題決定。	3
strmatch	startswith / 相等比較	先確認要字首還是整字相等；不要假裝兩者都等於舊函式預設。	26

原附錄印203 / PDF227短清單；原拼字sortrow在第二表註明。API見文末[1]-[24]；需要哪個軸、分母與排序必須由當題資料決定。

原書50項函式：文字、圖形、錯誤與時間

原名保留便於查原書；下表是查找方向，不宣稱所有預設、型別、缺值或版本等價。「相關題」連結概念，不保證該題示範了此API；只有正文程式區實際顯示的寫法才在本版執行。

原MATLAB名	Python方向	用途與必要差異	相關題
regexp	re.search / findall / fullmatch	分清查子字串、所有匹配或整字格式；回傳物件不同。	26
regexprep	re.sub	依模式替換文字；不用執行文字內容。	26
plot	Axes.plot	帶日期的路徑；指定軸、單位與圖例。	32
hist	Axes.hist	按區間分箱；MATLAB舊hist的組中心不能直接當Python的bins邊界。	32
bar	Axes.bar	類別 / 每日金額的柱形，正負結果對到0基準。	32
scatter	Axes.scatter	比較一對數值變數，不把點序自動當時間。	32
try	try:	開始可預期失敗的操作；不要把所有錯誤吞掉。	27
catch	except 已指定例外:	明確回報並處理原因，不轉成虛構績效。	27
circshift	np.roll / 時間滯後另用shift	roll循環繞回，不能直接作可用的前收。	28
datestr	Timestamp.strftime	格式化成文字；日期格式符號不同，不等於解析。	29
datenum	明定來源後轉Timestamp	MATLAB序列日不是Unix秒或pandas奈秒；本例直接解析ISO日期。	29
isempty	a.size==0 / df.empty	全NaN但有格子仍非空；先辨型別。	25、27
isfinite	np.isfinite	排除NaN與無窮；有限負價格仍不符合本版正價約定。	25、27
isnan	np.isnan / pd.isna	按型別找缺值；找出來不代表可填0。	25
islogical	確認bool型別 / dtype	整數0與1不等於布林陣列；遮罩與索引別混用。	11、25
randperm	rng.permutation	位置由0起算；打亂不等於時間序列可用的測試切分。	22

原附錄印203 / PDF227短清單；原拼字sortrow在第二表註明。API見文末[1]-[24]；需要哪個軸、分母與排序必須由當題資料決定。

正文操作與資料表的Python方向

原正文操作	Python方向	先檢查的差異
log	np.log	逐元素自然對數；報酬先用正價格比，見17。
find	flatnonzero / nonzero	本例一維從0起算；多維線性索引有掃描順序差，見12。
cell array	list / dict / 必要時object陣列	不規則長度與數字 / 文字型別不能硬當規則數值矩陣，見16。
table / timetable	DataFrame + Index / DatetimeIndex	一般表可用一般列標籤；帶時間的表用日期索引。本版日期為列、股票為欄，見29。
synchronize	join / merge / reindex	先定inner / outer與日期唯一性；只對明示持續生效配置ffill，見30。
retime	resample + 指定彙整	美元損益sum、普通報酬連乘；子樣本不稱整月，見31。
自建函式庫	def與明確輸入 / 回傳約定	先驗參數、維度、筆數與資料定義，再重用，見27。

原碼不是逐字替換

原書印200的初始化文字與程式有黏行；印201把z的取值誤稱y；印203列sortrow而非sortrows。本版按資料物件與正式函式意思重寫，不讓印刷錯誤變成Python規則。原書先刪x的第一列、只展示轉置而不覆寫x，後續log仍用刪後原方向；本版分別命名避免狀態跳躍。

NumPy陣列先用ndarray，不因接近MATLAB就改用舊matrix類別。普通轉置、共軛轉置與reshape不是同一動作；日期、股票與數字不能靠形狀相同就宣稱已對齊。

工具箱要換成哪一種Python工作？

作者在2021版列下列MATLAB工具箱用途，並提當時家用價格約每套50美元。本版只保留為當年說法，未查現行價格、授權或可用性，也不要求購買。Python是分開選資料結構、模型、圖形與執行工具，不是換一個套件名稱就已完成交易系統。

作者列的MATLAB用途	Python工作方向	本附錄沒有替你完成什麼
Optimization / Global Optimization	SciPy optimize的局部 / 全域求解器[16]	仍需目標、限制、尺度與收斂核對；不是自動找到獲利策略。
Statistics and Machine Learning	NumPy / SciPy統計與scikit-learn模型[18]	不自動處理未來資料洩漏、樣本選擇與交易成本。
Deep Learning	先定資料與學習問題，再選框架	本版未選框架、安裝或執行深度模型，不冒稱已有等價範例。
Signal Processing	SciPy signal及必要數學工具	不把指標參數或訊號處理函式當已驗證交易規則。
Financial（回測）	NumPy / pandas支援資料與帳務	不是現成完整回測系統；仍需訊號、成交、現金及費用約定。
Financial Instruments	先定契約、定價模型與反解問題	本版未選選擇權定價庫或聲稱已核對隱含波動率。
Econometrics	statsmodels時間序列工具[17]	ARIMA / VAR可按文件找介面；GARCH需另核所選實作，不全當一套件。
Datafeed	依資料商格式、授權與時間規則接資料	不連線下載、購買或暗示任一服務免費 / 當前可用。
Trading	依券商正式API與個別授權設計	不開戶、連真帳戶或下單；有Python語法不代表已可實盤。

原書也提James LeSage的Econometrics工具箱、Kevin Murphy的Bayes Net工具箱、使用者社群及作者網站函式庫。它們是歷史內容，不代表本版下載或驗證了當前版本；移植既有函式仍須核輸入、輸出、數學及授權。

原書、資料與驗證範圍

原書來源

Chan, Ernest P. Quantitative Trading: How to Build Your Own Algorithmic Trading Business. 第二版，Wiley，2021。附錄A Quick Survey of MATLAB，印199-204 / PDF223-228，正文、程式例、50項函式短清單、table / timetable、工具箱及自建函式均涵蓋。本版圖形與程式為原創改編，不複製書頁。

平台說明與本版選擇

作者描述MATLAB的編輯器、除錯器、內建數學、文字、圖形及部署能力，並認為多股票回測較試算表方便。本版用腳本或notebook組織Python程式，按陣列運算、日期資料表與函式檢查建立對應能力；未測兩平台速度、部署、編譯或能否取代所有商用功能。

資料與數字

AAPL、MSFT保存日資料中2024/1/2-1/9的Close支援本版算術。美分資料可重算展示值，但不證明當時訊號可得、實際成交、資料商時效或市場因果。配置金額、股數、買入估價與費用假設分開標示；沒有真實委託、借款或資料採購。

驗證的界線

顯示程式在分開的乾淨Python process執行並保存輸出；另以不同Decimal / 列表實作核對重要數值、形狀、索引、日期及分母，兩者仍共用保存價格與假設。完整討論答案只留私人紀錄。程式、版面通過不等於教師授課驗收、實盤系統或獲利效果。

官方文件與實作查核來源

查閱2026/10/19。文件支援API語義，不替本例提供市場績效證據；stable網址可能持續更新，本版執行版本列在第2頁。

[1] NumPy：給MATLAB使用者的差異說明

https://numpy.org/doc/stable/user/numpy-for-matlab-users.html

[2] NumPy 2.2：索引與切片

https://numpy.org/doc/2.2/user/basics.indexing.html

[3] NumPy 2.2：broadcasting

https://numpy.org/doc/2.2/user/basics.broadcasting.html

[4] NumPy 2.2：reshape

https://numpy.org/doc/2.2/reference/generated/numpy.reshape.html

[5] NumPy 2.2：std及ddof

https://numpy.org/doc/2.2/reference/generated/numpy.std.html

[6] NumPy 2.2：Generator亂數介面

https://numpy.org/doc/2.2/reference/random/generator.html

[7] NumPy 2.2：round的中點規則

https://numpy.org/doc/2.2/reference/generated/numpy.round.html

[8] NumPy 2.2：集合運算

https://numpy.org/doc/2.2/reference/routines.set.html

[9] Python 3.12：re文字模式

https://docs.python.org/3.12/library/re.html

[10] Python 3.12：例外處理

https://docs.python.org/3.12/tutorial/errors.html

[11] pandas 2.2.3：DataFrame.join

https://pandas.pydata.org/pandas-docs/version/2.2.3/reference/api/pandas.DataFrame.join.html

[12] pandas 2.2.3：Series.resample

https://pandas.pydata.org/pandas-docs/version/2.2.3/reference/api/pandas.Series.resample.html

官方文件與實作查核來源

查閱2026/10/19。文件支援API語義，不替本例提供市場績效證據；stable網址可能持續更新，本版執行版本列在第2頁。

[13] Matplotlib：圖形種類與輸入

https://matplotlib.org/stable/plot_types/index.html

[14] pandas 2.2.3：標籤與位置索引

https://pandas.pydata.org/pandas-docs/version/2.2.3/user_guide/indexing.html

[15] SciPy：稀疏矩陣部分特徵對eigs

https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.eigs.html

[16] SciPy：最佳化工具與求解條件

https://docs.scipy.org/doc/scipy/reference/optimize.html

[17] statsmodels：時間序列分析範圍

https://www.statsmodels.org/stable/tsa.html

[18] scikit-learn：機器學習工具範圍

https://scikit-learn.org/stable/

[19] Python 3.12：內建eval的執行語義

https://docs.python.org/3.12/library/functions.html

[20] NumPy 2.2：陣列形狀與接合

https://numpy.org/doc/2.2/reference/routines.array-manipulation.html

[21] MathWorks：std分母與維度參數

https://www.mathworks.com/help/matlab/ref/double.std.html

[22] MathWorks：round預設及版本差異

https://www.mathworks.com/help/matlab/ref/double.round.html

[23] MathWorks：synchronize日期合併

https://www.mathworks.com/help/matlab/ref/synchronize.html

[24] MathWorks：retime分組、標籤與彙整

https://www.mathworks.com/help/matlab/ref/timetable.retime.html